

第2章 运算符与表达式

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 C++ 全部运算符,说出与 C 的对应关系
- 理解溢出:有符号 UB、unsigned 回绕、整数除零也是 UB
- 理解短路、三目、复合赋值的真实语义
- 掌握位运算生产场景:标志位、掩码、快速乘除 2 的幂
- 掌握浮点比较的正确姿势,理解 NaN 的坑
- 会用 C++20 三路比较与 auto 比较,写出可读的比较逻辑

💡 从 C 到 C++:本章主题如何迁移

运算符是 C++ 从 C 继承最彻底的部分:算术、位、逻辑、比较、赋值几乎 1:1 保留,坑也全部保留:有符号溢出仍是 UB,求值顺序仍未规定。C++20 引入三路比较,把 strcmp 式的"负数/零/正数"约定提升为语言级语义。

学完本章你会形成两个习惯:把"会不会是 UB"当作写每行表达式的默认提问;把复杂比较交给三路比较与标准库。这正是 C 工程师进阶 C++ 的分水岭。

📦 前置知识自查

- C 的算术运算符、除法截断与取模符号规则
- C 的位运算符、逻辑运算符与短路求值
- sizeof、逗号表达式、三目运算符 ?:
- C 的运算符优先级表(乘法高于加法、移位低于算术)
- 头文件与基本 I/O

🚀 本章学习路线

1. 第一阶段(熟悉):§2.1-§2.2 算术与自增自减,理解"UB 文化"在运算符上的体现
2. 第二阶段(重点):§2.3-§2.5 位运算、短路、比较,生产代码出错率最高的区域
3. 第三阶段(进阶):§2.6-§2.8 三路比较、优先级表、求值顺序,做自测题与章末实验
4. 验收标准:能背优先级口诀,能解释 `i++ + i++` 是 UB,能写标志位权限代码

本章知识导图

- §2.1 算术:溢出(UB vs 回绕)、除法截断、负数取模
- §2.2 自增自减:前置/后置语义、++ 是表达式
- §2.3 位运算:全套运算符、标志位、有符号右移
- §2.4 逻辑与短路:&& || !、短路保证、& 与 && 之别
- §2.5 比较:值比较、浮点相等、NaN
- §2.6 三路比较:C++20 的 <=> 与 auto 比较
- §2.7 优先级与求值顺序:优先级表、未定序 UB
- §2.8 三目与复合赋值?: 类型规则、+= 只求值一次

§2.1 算术运算:溢出、除法与取模

C++ 的算术运算符 + - * / % 与 C 完全一致,行为不变。下面这张对照表是本章最重要的一张表。

C 的写法/行为	C++ 的写法/行为	差异
有符号溢出:UB	有符号溢出:UB	完全相同,编译器可做任意优化
unsigned 回绕	unsigned 回绕	相同,按模 2^N 运算,有定义
整数除法向零截断	整数除法向零截断	相同(C99 起);负数 $-7/2 == -3$
取模符号同被除数	取模符号同被除数	相同; $-7 \% 3 == -1$
整数除零:UB	整数除零:UB	相同,不是异常,程序行为未定义
无幂运算符	无幂运算符	相同;2 的幂用 <<,其他用 std::pow

有符号溢出是 UB,编译器认为"这不可能发生"并据此优化:优化器可能把 $x + 1 > x$ 直接判定为恒真。生产规则:可能溢出的运算先检查边界或用 int64_t 兜底,并用 UBSan 扫描。unsigned 回绕定义明确: $0xFFFFFFFF + 1$ 得到 0,协议序号、校验和都依赖它。

⚠ 易错点 1:整数除零是 UB,不是异常

C/C++ 的整数除零不会抛异常、不会返回 0,行为未定义——常见结果是程序崩溃,但标准不保证任何结果。浮点除零则安全:得到 inf 或 NaN。生产规则:除数来自外部输入时先判零再运算。

示例 2-1 溢出、回绕与除法取模

```

#include <iostream>
#include <climits>

int main() {
    // unsigned 回绕:有定义,按模 2^32 计算
    unsigned u = 0xFFFFFFFFu;
    std::cout << "回绕: " << (u + 1) << "\n";      // 0

    // 有符号溢出:UB,勿依赖
    int imax = INT_MAX;
    std::cout << "有符号溢出(UB,勿依赖): " << (imax + 1) << "\n";

    // 除法向零截断,取模符号同被除数
    std::cout << "-7 / 2 = " << (-7 / 2) << "\n";    // -3
    std::cout << "-7 % 3 = " << (-7 % 3) << "\n";    // -1
    std::cout << "7 % -3 = " << (7 % -3) << "\n";    // 1

    // 整数除零:UB,仅注释展示,勿执行
    // int bad = 100 / 0;
    return 0;
}

```

注意与 Python 的区别:Python 的 % 结果与除数同号(-7 % 3 == 2),C/C++ 与被除数同号,跨语言移植取模代码要显式处理。

§2.2 自增自减:前置与后置

C 的写法	C++ 的写法	差异
i++ 后置,返回旧值	i++ 后置,返回旧值	完全相同
++i 前置,返回新值	++i 前置,返回新值	完全相同
++ 是表达式	++ 是表达式	相同;Go 中 ++ 是语句,不可嵌套
i++++ 不合法	i++++ 不合法	相同,后置自增的结果不是左值

自增自减与 C 完全一致,记住三点:后置返回旧值的拷贝;后置结果不是左值,i++ = 0 不合法;单独成句时前后缀等价。生产习惯:循环统一写 ++i——类类型上后置自增会构造临时对象,习惯统一可避免将来泛型化踩坑。

示例 2-2 前置与后置的区别

```

#include <iostream>

int main() {
    int i = 0;
    int a = i++;      // a = 0, 然后 i = 1(后置先取旧值)
    int b = ++i;      // i = 2, 然后 b = 2(前置先自增)
    std::cout << "a = " << a << ", b = " << b << ", i = " << i << "\n";

    // 自增结果不是左值,以下写法不合法
    // i++ = 5;
    return 0;
}

```

§2.3 位运算:全套运算符与生产场景

C++ 提供与 C 相同的六种位运算:& | ^ ~ << >>。与 Java/JS 不同,C++ 没有 >>>:无符号类型右移天然就是逻辑移位(高位补零);有符号类型 C++20 起明确规定是算术移位,C++17 及以前是实现定义,但主流编译器早已统一为算术移位。

C 的写法	C++ 的写法	差异
& ^ ~ << >>	& ^ ~ << >>	运算符完全相同
有符号右移:实现定义	有符号右移:C++20 定为算术移位	C++20 起语义确定
无 >>>	无 >>>	与 Java/JS 不同;无符号右移用无符号类型
位运算结果类型	位运算结果类型	相同:整型提升后运算,无符号溢出回绕

⚠ 易错点 2:有符号类型的移位陷阱

对负数右移是算术移位(向下取整的除法):-8 >> 1 == -4。负数左移曾是 UB,即便 C++20 已定义其结果,位运算也只对无符号类型做。~x == -(x+1),~0 == -1,别把 ~ 当逻辑非。

生产场景:标志位与权限集合是位运算最常见的用途——| 组合、& 判断、^ 切换、~ 掩码。还常用于哈希、位图、协议解析与快速乘除 2 的幂。注意:位运算不做短路,两个操作数都会求值。

示例 2-3 权限标志位的实现

```
#include <iostream>

// 权限位:每比特一个权限(C 的老办法,C++ 依然常用)
constexpr unsigned kRead    = 1u << 0;    // 0b001
constexpr unsigned kWrite   = 1u << 1;    // 0b010
constexpr unsigned kExecute = 1u << 2;    // 0b100

int main() {
    unsigned perm = kRead | kWrite;        // 组合权限
    std::cout << (perm & kRead) << "\n";    // 非零:有读权限
    std::cout << (perm & kExecute) << "\n"; // 0:无执行权限

    perm ^= kWrite;                        // 切换:去掉写权限
    std::cout << (perm & kWrite) << "\n";    // 0

    perm |= kExecute;                      // 加执行权限
    perm &= ~kRead;                        // 掩码去掉读权限
    std::cout << (perm & (kExecute | kRead)) << "\n"; // 只余执行位

    // 移位即乘除 2 的幂:x<<2==x*4,x>>1==x/2(无符号)
    unsigned x = 7;
    std::cout << (x << 2) << " " << (x >> 1) << "\n"; // 28 3
    return 0;
}
```

§2.4 逻辑运算与短路求值

&& 与 || 的短路语义与 C 完全一致:左侧决定结果时,右侧不再求值。短路是语言保证而非优化,它让"指针非空再解引用"安全成立。小差异:逻辑表达式结果类型是 bool。

C 的写法	C++ 的写法	差异
a && b 结果是 int	a && b 结果是 bool	C++ 结果类型是 bool
短路求值	短路求值	完全相同,语言保证
if (p) 指针非零即真	if (p) 指针非零即真	相同;条件上下文允许指针与整数
& 与 && 是不同运算符	& 与 && 是不同运算符	相同;常被误用

⚠ 易错点 3:用 & 代替 && 会丢失短路

if (p && p->x) 安全,if (p & p->x) 是灾难:& 先求值两侧,空指针解引用直接 UB。a & b 的结果是数值而非真值,评审中把 & 写成 && 是高频错误;两者优先级不同(& 高于 &&),混用要加括号。

示例 2-4 短路求值的两种场景

```
#include <iostream>

int main() {
    int n = 0;
    // 短路:左侧 false,右侧 10/n 不执行,不会除零
    bool safe = (n != 0) && (10 / n > 1);
    std::cout << "safe = " << safe << "\n";    // 0

    // || 左侧为 true 时右侧不求值:常用于给默认值
    const char* env = nullptr;
    const char* value = env ? env : "默认值";    // 判空兜底
    std::cout << value << "\n";

    // 副作用:短路使右侧副作用不发生
    int cnt = 0;
    bool r = (n != 0) || (++cnt != 0);    // 右侧求值,cnt 变 1
    bool r2 = (n == 0) || (++cnt != 0);    // 左侧已 true,右侧不求值
    std::cout << "cnt = " << cnt << ", r = " << r << ", r2 = " << r2 << "\n";
    return 0;
}
```

§2.5 比较运算与浮点相等

== != < <= > >= 都是值比较,与 C 相同:数组名比较的是地址,结构体不能整体比较(C++20 可自动生成,见下节)。浮点相等是经典陷阱:0.1 + 0.2 != 0.3;NaN 不等于任何值,包括它自己。

C 的写法	C++ 的写法	差异
<code>a == b</code> 值比较	<code>a == b</code> 值比较	完全相同
NaN 不等于任何值	NaN 不等于任何值	相同;判断 NaN 用 <code>isnan()</code>
浮点相等直接 <code>==</code>	浮点相等直接 <code>==</code>	相同;都需要 <code>epsilon</code> 或整数化
结构体不能 <code>==</code>	结构体不能 <code>==(C++20 可自动生成)</code>	C++20 用 <code><=></code> 或默认比较

⚠ 易错点 4:浮点相等比较

if (`a == b`) 对浮点数几乎总是错的:`0.1 + 0.2 == 0.3` 是 `false`。正确姿势:误差允许时用阈值比较;能整数化就整数化(金额用分、距离用毫米)。生产规范:浮点相等比较必须写注释说明误差来源。

示例 2-5 浮点比较的正确姿势

```
#include <iostream>
#include <cmath>

// 误差阈值按数量级缩放,不用绝对 1e-9
bool nearlyEqual(double a, double b, double eps = 1e-9) {
    return std::fabs(a - b) <= eps * std::max({1.0, std::fabs(a), std::fabs(b)});
}

int main() {
    double x = 0.1 + 0.2;
    std::cout << (x == 0.3) << "\n";           // 0:经典陷阱
    std::cout << nearlyEqual(x, 0.3) << "\n"; // 1:阈值比较
    std::cout << (x < 0.3) << "\n";           // 1:0.30000000000000004

    // NaN 不等于任何值,包括它自己
    double nan = std::nan("");
    std::cout << (nan == nan) << "\n";         // 0
    std::cout << std::isnan(nan) << "\n";     // 1:判断 NaN 用 isnan

    // 能整数化就整数化:金额用"分"存整数
    long long cents = 125;                     // 1.25 元
    std::cout << (cents / 100.0) << "\n";     // 展示时才除 100
    return 0;
}
```

§2.6 C++20 三路比较运算符:告别 strcmp 时代

C 时代比较两个对象要靠 `strcmp`/自己的 `compare` 函数,返回负数/零/正数,调用方再写 if (`cmp < 0`)。C++20 的三路比较运算符 `<=>`(星船运算符)把"三态比较"提升为语言语义:`a <=> b` 返回序类型(`strong_ordering` / `partial_ordering` / `weak_ordering`),结果要么是序值要么是相等值,零比较的正确性由语言保证。

C 的写法	C++ 的写法	差异
strcmp(a, b) 返回 <0/0/>0	a <=> b 返回序类型	C++ 把约定变成类型
手写 6 个比较函数	= default 自动生成 6 个	C++20 声明式比较
用 memcmp 比较结构体	结构体直接 == <	C++ 按成员逐字段比较
无	float 比较得 partial_ordering	NaN 参与时是"不可比较",安全

示例 2-6 三路比较自动生成全部比较运算符

```
#include <iostream>
#include <compare>          // C++20:std::strong_ordering 等

// = default:自动生成六个比较运算符
struct Version {
    int major;
    int minor;
    auto operator<=>(const Version&) const = default;
};

int main() {
    Version v1{2, 10};
    Version v2{2, 9};

    // 直接比较,不用再写 compare
    std::cout << (v1 < v2) << (v1 == v2) << (v1 > v2) << "\n"; // 0 0 1

    // 三路比较的返回:序类型,而非 int
    std::strong_ordering ord = v1 <=> v2;
    std::cout << (ord < 0) << (ord == 0) << (ord > 0) << "\n"; // 0 0 1

    // 浮点三路比较:partial_ordering,NaN 参与时不可比较
    double a = 0.3, b = 0.1 + 0.2;
    std::partial_ordering po = a <=> b;
    std::cout << (po == std::partial_ordering::equivalent) << "\n"; // 0
    return 0;
}
```

生产建议:C++20 项目中,按字典序比较的纯数据类直接声明 `auto operator<=>` 并 `= default`,六个比较运算符自动一致。C++17 及以下用 `friend` 函数或宏生成。注意:裸数组不能自动比较,`std::array` 可以;含指针成员的结构体比较的是地址。

§2.7 优先级、结合性与求值顺序

C++ 的优先级表与 C 相同,口诀:括号后缀、一元、乘除加减、移位、关系相等、位与异或或、逻辑与或、三目、赋值、逗号。

优先级	运算符(同格内结合性从左到右)
1(最高)	() 下标 . -> ++ -- (后缀) ::
2	! ~ + - * & (一元) ++ -- (前缀) (类型)强转 sizeof new
3	* / %
4	+ -
5	<< >>
6	< <= > >=
7	== !=
8	&
9	^
10	
11	&&
12	
13	?:
14	= += -= *= /= %= <<= >>= &= = ^=
15(最低)	, 逗号运算符

优先级决定"谁先结合",求值顺序决定"谁先算出值",两者无关。C++ 与 C 相同:操作数与函数实参的求值顺序大多未规定。i++ + i++ 两次自增读写同一变量且无顺序点,是 UB;f(a(), b()) 的实参调用顺序也未规定(Java、C# 则从左到右)。生产规则:一条语句只做一件事,同一变量在一个表达式里最多一个副作用。



易错点 5:未定序求值是 UB 的重灾区

printf("%d %d", i, i++); 在 C 里常见,在 C++ 里同样是 UB:实参求值顺序未规定。即使输出"看起来稳定",编译器也可任意重排。修复:拆成两条语句,先自增再传值。

示例 2-7 求值顺序:UB 与正确写法

```

#include <iostream>

int sideEffect(int v) {
    std::cout << "调用 sideEffect(" << v << ")\n";
    return v;
}

int main() {
    // UB:两次自增未定序,程序行为未定义
    // int x = 1;
    // int r = x++ + x++;

    // 同样危险:实参求值顺序未规定
    // std::cout << sideEffect(1) + sideEffect(2) << "\n";

    // 正确:拆成语句,顺序明确
    int x = 1;
    int a = x++;
    int b = x++;
    std::cout << "a + b = " << (a + b) << "\n";    // 1 + 2 = 3

    int s = sideEffect(1);
    s += sideEffect(2);
    std::cout << "s = " << s << "\n";
    return 0;
}

```

§2.8 三目运算符与复合赋值

?: 与复合赋值都从 C 原样继承。三目是表达式,分支类型要一致或可隐式转换,长表达式务必加括号。复合赋值的要点:左操作数只求值一次—— $a[i++] += 1$ 安全,而 $a[i++] = a[i++] + 1$ 求值两次,是未定义行为。能用复合赋值就不用展开写法。

示例 2-8 三目与复合赋值

```
#include <iostream>

int main() {
    // 三目:两个分支类型一致时是表达式
    int score = 61;
    const char* verdict = (score >= 60) ? "及格" : "不及格";
    std::cout << verdict << "\n";

    // 复合赋值:左操作数只求值一次
    int arr[3] = {10, 20, 30};
    int i = 0;
    arr[i++] += 5;      // 只对 i++ 求值一次:arr[0] = 15,i = 1
    std::cout << arr[0] << " " << i << "\n";

    // 三目可嵌套但可读性差,生产不这么写
    int grade = 85;
    const char* level = grade >= 90 ? "优" : grade >= 75 ? "良" : "差";
    std::cout << level << "\n";
    return 0;
}
```

本章速查表

主题	速查内容
有符号溢出	UB;先检查边界或 int64_t;UBSan 检测
unsigned 回绕	有定义,模 2^N ;序号、校验和放心用
整数除零	UB,不是异常;浮点除零得 inf/NaN
除法/取模	向零截断;取模符号同被除数($-7\%3=-1$)
自增自减	$++i$ 返回新值, $i++$ 返回旧值;结果非左值
位运算	$\& ^ \sim < < > >$;无 $> > >$;右移用无符号
标志位	$ $ 组合、 $\&$ 判断、 $^$ 切换、 $\sim$ 掩码;常量 $1u << n$
短路	$\&\& $ 保证短路;别用 $\&$ 代替 $\&\&$
逻辑结果	$\&\& !$ 返回 bool;条件上下文允许指针
浮点相等	禁止 $==$;用相对误差阈值或整数化
NaN	不等于任何值;用 std::isnan 判断
三路比较	C++20:operator<=> = default 生成 6 个比较
优先级	后缀>一元>乘除>加减>移位>关系>相等> $\&>^> >\&\&> >?:>$ 赋值>逗号
求值顺序	操作数顺序未规定;一表达式内同变量只一个副作用
复合赋值	左操作数只求值一次, $a[i++] += 1$ 安全

最小必会清单

- 能解释有符号溢出是 UB、unsigned 回绕,并给出检测手段
- 能说出 $-7 / 2$ 与 $-7 \% 3$ 的值及与 Python 取模的差异

- 能写出权限标志位的组合/判断/切换/掩码代码
- 能说明短路求值的语言保证与 `&` 和 `&&` 的差别
- 能写出浮点比较的正确函数,能解释 NaN 的 `==` 行为
- 能用 `<=> = default` 让结构体获得全部比较运算符
- 能背出优先级口诀,能解释 `i++ + i++` 为何是 UB
- 能说明复合赋值为何只求值一次

自测题

Q `int i = 1; 表达式 i++ + i++ 的值是多少?`

✓ 参考答案

没有答案——这是未定义行为:两次自增作用于同一变量且未定序,编译器可任意重排。正确回答是"UB,不能这么写"。

Q 写出 `-7 / 2`、`-7 % 3`、`7 % -3` 在 C++ 中的结果,并说明取模符号规则。

✓ 参考答案

分别是 -3、-1、1。整数除法向零截断,取模符号与被除数一致。Python 中 `-7 % 3 == 2`(与除数同号),移植时要注意。

Q `bool a = 1 && 1; bool b = 1 & 1; bool c = (2 < 1) || (1 < 2);` 三个变量分别是多少?`&` 与 `&&` 的关键区别是什么?

✓ 参考答案

三者都是 true。区别:`&&` 短路且结果为 bool;`&` 不短路、结果是整数。指针判空后用 `&` 会解引用空指针。

Q `double a = 0.1 + 0.2;` 判断 `a == 0.3` 会得到什么?给出两种正确的比较方案。

✓ 参考答案

得到 false:0.1 与 0.2 在二进制浮点中都是近似值。方案一:相对误差阈值比较;方案二:整数化——金额用分、距离用毫米,全程整数运算,展示时才转浮点。

Q 给 Version 结构体(含 major、minor 两个 int 成员)声明比较运算符,使 `v1 < v2`、`v1 == v2`、`v1 > v2` 都能编译。

✓ 参考答案

结构体内写 `auto operator<=>(const Version&) const = default;` 一个声明自动生成六个比较运算符,按成员声明顺序逐字段比较。

章末实验题:表达式求值计算器

实验 2:表达式求值计算器

实验目标:写一个命令行程序:读入表达式,用"中缀转后缀(调度场算法)+ 栈求值"输出结果,覆盖全章运算符知识点。

实验要求:

- 任务 1:词法器把输入切成数字与运算符,支持 $+$ $-$ $*$ $/$ $\%$ $\&$ $|$ $^$ $\sim$ $<<$ $>>$ $\&\&$ $||$ $!$ $==$ $!=$ $<$ $<=$ $>$ $>=$ 与括号(覆盖:算术/位/逻辑/比较/一元运算符)
- 任务 2:用 `std::map` 存优先级表,数字越大越优先(覆盖:优先级、`map`)
- 任务 3:调度场算法转后缀,处理括号与一元负号(覆盖:结合性;后缀消除未定序求值)
- 任务 4:用 `std::vector` 当栈做后缀求值(覆盖:栈、`int64_t` 与回绕)
- 任务 5:main 演示短路、三目、自增、负数取模与回绕,解释 $++$ 与 $?:$ 为何不进求值器(覆盖:短路/三目/自增/取模/溢出)
- 任务 6:手动验证: $3 + 4 * 2$ 得 11; $(1 < 4) - 6$ 得 10; $-7 \% 3$ 得 -1
- 任务 7:用 `-Wall -Wextra` 编译,全程零警告

实验环境:任意 C++17 编译器。编译命令:`g++ -std=c++17 -Wall -Wextra calc.cpp -o calc`

第一步 写词法器与优先级表,编译通过后再写调度场

第二步 写求值器,先支持 $+$ $-$ $*$ $/$ 与括号,验证无误后加位运算与逻辑运算

第三步 后写 main 演示,运行算例与手算核对

✅ 参考答案(第一部分:优先级表与词法器)

```
// 表达式求值计算器(第一部分:词法器)
#include <iostream>
#include <string>
#include <vector>
#include <map>
#include <cctype>

// 任务 2:优先级表,数字越大越优先;NEG 是最高优先级的一元负号
std::map<std::string, int> kPrec = {
    {"||", 1}, {"&&", 2},
    {"|", 3}, {"^", 4}, {"&", 5},
    {"==", 6}, {"!=", 6},
    {"<", 7}, {"<=", 7}, {">", 7}, {">=", 7},
    {"<<", 8}, {">>", 8},
    {"+", 9}, {"-", 9},
    {"*", 10}, {"/", 10}, {"%", 10},
    {"NEG", 100}, {"!", 100}, {"~", 100},
};

bool isOp(const std::string& t) { return kPrec.count(t) > 0; }

// 任务 1:词法器;'!' '~' 与"需要操作数处的 -"作一元前缀运算符
std::vector<std::string> tokenize(const std::string& expr) {
    std::vector<std::string> out;
    for (std::size_t i = 0; i < expr.size(); ) {
        char c = expr[i];
        if (std::isspace(static_cast<unsigned char>(c))) { ++i; continue; }
        if (std::isdigit(static_cast<unsigned char>(c))) {
            std::size_t j = i;
            while (j < expr.size() &&
                std::isdigit(static_cast<unsigned char>(expr[j]))) ++j;
            out.push_back(expr.substr(i, j - i));
            i = j;
            continue;
        }
        std::string two = expr.substr(i, 2);
        if (isOp(two)) { out.push_back(two); i += 2; continue; }
        if (c == '-' && (out.empty() || out.back() == "(" || isOp(out.back()))) {
            out.push_back("NEG"); ++i; continue; // 一元负号
        }
        out.push_back(std::string(1, c)); // 单字符运算符或括号
        ++i;
    }
    return out;
}
```

✅ 参考答案(第二部分:调度场转换与栈求值)

```
// 第二部分:中缀转后缀(调度场算法)
// 任务 3:调度场算法。后缀表达式的运算顺序确定,天然消除未定序 UB
std::vector<std::string> toRpn(const std::vector<std::string>& tokens) {
    std::vector<std::string> out, ops;
    for (const std::string& t : tokens) {
        if (t == "(") { ops.push_back(t); continue; }
        if (t == ")") {
            while (!ops.empty() && ops.back() != "(") {
                out.push_back(ops.back());
                ops.pop_back();
            }
            if (!ops.empty()) ops.pop_back(); // 弹出 '('
            continue;
        }
        if (isOp(t)) {
            if (t == "NEG" || t == "!" || t == "~") {
                ops.push_back(t); // 前缀运算符:直接压栈
            } else {
                while (!ops.empty() && ops.back() != "(" &&
                    kPrec[ops.back()] >= kPrec[t]) {
                    out.push_back(ops.back());
                    ops.pop_back();
                }
                ops.push_back(t); // 同优先级左结合:先弹后压
            }
            continue;
        }
        out.push_back(t); // 数字
    }
    while (!ops.empty()) { out.push_back(ops.back()); ops.pop_back(); }
    return out;
}
```

✅ 参考答案(第三部分:栈求值)

```
// 任务 4:栈求值。int64_t 计算,位运算按无符号语义回绕
long long applyOp(long long a, long long b, const std::string& op) {
    switch (op[0]) {
        case '+': return a + b;
        case '-': return a - b;
        case '*': return a * b;
        case '/': return b == 0 ? 0 : a / b;    // 除零防御,生产应报错
        case '%': return b == 0 ? 0 : a % b;    // 取模符号同被除数
    }
    if (op == "&") return a & b;
    if (op == "|") return a | b;
    if (op == "^") return a ^ b;
    if (op == "<<") return a << b;
    if (op == ">>") return a >> b;
    if (op == "&&") return (a != 0 && b != 0) ? 1 : 0;
    if (op == "||") return (a != 0 || b != 0) ? 1 : 0;
    if (op == "==") return a == b;
    if (op == "!=") return a != b;
    if (op == "<") return a < b;
    if (op == "<=") return a <= b;
    if (op == ">") return a > b;
    if (op == ">=") return a >= b;
    return 0;
}

long long evalRpn(const std::vector<std::string>& rpn) {
    std::vector<long long> st;
    for (const std::string& t : rpn) {
        if (!isOp(t)) { st.push_back(std::stoll(t)); continue; }
        if (t == "NEG") { st.back() = -st.back(); continue; }
        if (t == "!") { st.back() = st.back() == 0 ? 1 : 0; continue; }
        if (t == "~") { st.back() = ~st.back(); continue; }
        long long b = st.back(); st.pop_back();
        long long a = st.back(); st.pop_back();
        st.push_back(applyOp(a, b, t));
    }
    return st.back();
}
```

✅ 参考答案(第四部分:main 主流程与知识点演示)

```
// 第四部分:main 主流程
int main() {
    std::string line;
    std::cout << "输入表达式(如 3 + 4 * 2): ";
    std::getline(std::cin, line);

    std::vector<std::string> tokens = tokenize(line);
    std::vector<std::string> rpn = toRpn(tokens);
    std::cout << "后缀: ";
    for (const std::string& t : rpn) std::cout << t << " ";
    std::cout << "\n结果: " << evalRpn(rpn) << "\n";

    // 任务 5:短路演示(&& 左侧为假,右侧不求值)
    int n = 0;
    bool safe = (n != 0) && (10 / n > 1);
    std::cout << "短路演示: " << safe << "\n";

    // 任务 5:负数取模与回绕(与 C 完全一致)
    std::cout << "-7 % 3 = " << (-7 % 3) << "\n";
    unsigned mx = 0xFFFFFFFFu;
    std::cout << "unsigned 回绕: " << (mx + 1) << "\n";

    // 任务 5:三目与前置自增(求值器语法不含 ?: 与 ++:
    // 前者需三操作数,后者是赋值语义,工程求值器同样如此)
    for (int i = 0; i < 3; ++i) {
        std::cout << "第" << i << "次:" << (i == 2 ? "末位" : "普通") << " ";
    }
    std::cout << "\n";
    return 0;
}
```

设计说明:调度场算法在转后缀时"按优先级出栈",最终运算顺序完全确定——这是生产环境规避求值顺序 UB 的惯用手法。求值用 long long;除零做了防御返回 0,生产代码应改为报错退出。

下一章预告:第3章 流程控制——if/else 的条件类型要求、switch 与 C 的差别、范围 for 循环、循环中的迭代器失效,以及 goto 在 C++ 中的正确归宿。